

they often succeed where a direct attack via Newton's method alone fails. The next section deals with these methods.

CITED REFERENCES AND FURTHER READING:

- Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 14.[1]
 Ostrowski, A.M. 1966, *Solutions of Equations and Systems of Equations*, 2nd ed. (New York: Academic Press).
 Ortega, J., and Rheinboldt, W. 1970, *Iterative Solution of Nonlinear Equations in Several Variables* (New York: Academic Press); reprinted 2000 (Philadelphia: S.I.A.M.).

9.7 Globally Convergent Methods for Nonlinear Systems of Equations

We have seen that Newton's method for solving nonlinear equations has an unfortunate tendency to wander off into the wild blue yonder if the initial guess is not sufficiently close to the root. A *global* method [1] would be one that converges to a solution from almost any starting point. Such global methods do exist for minimization problems; an example is the quasi-Newton method that we will describe in §10.9. In this section we will develop an algorithm that is an analogous quasi-Newton method for multidimensional root finding. Alas, while it is better behaved than Newton's method, it is still not truly global.

What the method *does* do is combine the rapid local convergence of Newton's method with a higher-level strategy that guarantees at least *some* progress at each step — either toward an actual root (usually), or else, hopefully rarely, toward the situation labeled “no root here!” in Figure 9.6.1. In the latter case, the method recognizes the problem and signals failure. By contrast, Newton's method can bounce around forever, and you are never sure whether or not to quit.

Recall our discussion of §9.6: The Newton step for the set of equations

$$\mathbf{F}(\mathbf{x}) = 0 \quad (9.7.1)$$

is

$$\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}} + \delta\mathbf{x} \quad (9.7.2)$$

where

$$\delta\mathbf{x} = -\mathbf{J}^{-1} \cdot \mathbf{F} \quad (9.7.3)$$

Here $\mathbf{J}$ is the Jacobian matrix. How do we decide whether to accept the Newton step $\delta\mathbf{x}$? A reasonable strategy is to require that the step decrease $|\mathbf{F}|^2 = \mathbf{F} \cdot \mathbf{F}$. This is the same requirement we would impose if we were trying to minimize

$$f = \frac{1}{2} \mathbf{F} \cdot \mathbf{F} \quad (9.7.4)$$

(The $\frac{1}{2}$ is for later convenience.) Every solution to (9.7.1) minimizes (9.7.4), but there may be local minima of (9.7.4) that are not solutions to (9.7.1). Thus, as already mentioned, simply applying one of our minimum-finding algorithms from Chapter 10 to (9.7.4) is *not* a good idea.

To develop a better strategy, note that the Newton step (9.7.3) is a *descent direction* for f :

$$\nabla f \cdot \delta \mathbf{x} = (\mathbf{F} \cdot \mathbf{J}) \cdot (-\mathbf{J}^{-1} \cdot \mathbf{F}) = -\mathbf{F} \cdot \mathbf{F} < 0 \quad (9.7.5)$$

Thus our strategy is quite simple: We always first try the full Newton step, because once we are close enough to the solution we will get quadratic convergence. However, we check at each iteration that the proposed step reduces f . If not, we *backtrack* along the Newton direction until we have an acceptable step. Because the Newton step is a descent direction for f , we are guaranteed to find an acceptable step by backtracking. We will discuss the backtracking algorithm in more detail below.

Note that this method minimizes f only “incidentally,” either by taking Newton steps designed to bring $\mathbf{F}$ to zero, or by backtracking along such a step. The method is *not* equivalent to minimizing f directly by taking Newton steps designed to bring ∇f to zero. While the method can nevertheless still fail by converging to a local minimum of f that is not a root (as in Figure 9.6.1), this is quite rare in real applications. The routine `newt` below will warn you if this happens. The only remedy is to try a new starting point.

9.7.1 Line Searches and Backtracking

When we are not close enough to the minimum of f , taking the full Newton step $\mathbf{p} = \delta \mathbf{x}$ need not decrease the function; we may move too far for the quadratic approximation to be valid. All we are guaranteed is that *initially* f decreases as we move in the Newton direction. So the goal is to move to a new point $\mathbf{x}_{\text{new}}$ along the *direction* of the Newton step $\mathbf{p}$, but not necessarily all the way:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}} + \lambda \mathbf{p}, \quad 0 < \lambda \leq 1 \quad (9.7.6)$$

The aim is to find λ so that $f(\mathbf{x}_{\text{old}} + \lambda \mathbf{p})$ has decreased sufficiently. Until the early 1970s, standard practice was to choose λ so that $\mathbf{x}_{\text{new}}$ exactly minimizes f in the direction $\mathbf{p}$. However, we now know that it is extremely wasteful of function evaluations to do so. A better strategy is as follows: Since $\mathbf{p}$ is always the Newton direction in our algorithms, we first try $\lambda = 1$, the full Newton step. This will lead to quadratic convergence when $\mathbf{x}$ is sufficiently close to the solution. However, if $f(\mathbf{x}_{\text{new}})$ does not meet our acceptance criteria, we *backtrack* along the Newton direction, trying a smaller value of λ , until we find a suitable point. Since the Newton direction is a descent direction, we are guaranteed to decrease f for sufficiently small λ .

What should the criterion for accepting a step be? It is *not* sufficient to require merely that $f(\mathbf{x}_{\text{new}}) < f(\mathbf{x}_{\text{old}})$. This criterion can fail to converge to a minimum of f in one of two ways. First, it is possible to construct a sequence of steps satisfying this criterion with f decreasing too slowly relative to the step lengths. Second, one can have a sequence where the step lengths are too small relative to the initial rate of decrease of f . (For examples of such sequences, see [2], p. 117.)

A simple way to fix the first problem is to require the *average* rate of decrease of f to be at least some fraction α of the *initial* rate of decrease $\nabla f \cdot \mathbf{p}$:

$$f(\mathbf{x}_{\text{new}}) \leq f(\mathbf{x}_{\text{old}}) + \alpha \nabla f \cdot (\mathbf{x}_{\text{new}} - \mathbf{x}_{\text{old}}) \quad (9.7.7)$$

Here the parameter α satisfies $0 < \alpha < 1$. We can get away with quite small values of α ; $\alpha = 10^{-4}$ is a good choice.

The second problem can be fixed by requiring the rate of decrease of f at $\mathbf{x}_{\text{new}}$ to be greater than some fraction β of the rate of decrease of f at $\mathbf{x}_{\text{old}}$. In practice, we will not need to impose this second constraint because our backtracking algorithm will have a built-in cutoff to avoid taking steps that are too small.

Here is the strategy for a practical backtracking routine: Define

$$g(\lambda) \equiv f(\mathbf{x}_{\text{old}} + \lambda \mathbf{p}) \quad (9.7.8)$$

so that

$$g'(\lambda) = \nabla f \cdot \mathbf{p} \quad (9.7.9)$$

If we need to backtrack, then we model g with the most current information we have and choose λ to minimize the model. We start with $g(0)$ and $g'(0)$ available. The first step is always the Newton step, $\lambda = 1$. If this step is not acceptable, we have available $g(1)$ as well. We can therefore model $g(\lambda)$ as a quadratic:

$$g(\lambda) \approx [g(1) - g(0) - g'(0)]\lambda^2 + g'(0)\lambda + g(0) \quad (9.7.10)$$

Taking the derivative of this quadratic, we find that it is a minimum when

$$\lambda = -\frac{g'(0)}{2[g(1) - g(0) - g'(0)]} \quad (9.7.11)$$

Since the Newton step failed, one can show that $\lambda \lesssim \frac{1}{2}$ for small α . We need to guard against too small a value of λ , however. We set $\lambda_{\min} = 0.1$.

On second and subsequent backtracks, we model g as a cubic in λ , using the previous value $g(\lambda_1)$ and the second most recent value $g(\lambda_2)$:

$$g(\lambda) = a\lambda^3 + b\lambda^2 + g'(0)\lambda + g(0) \quad (9.7.12)$$

Requiring this expression to give the correct values of g at λ_1 and λ_2 gives two equations that can be solved for the coefficients a and b :

$$\begin{bmatrix} a \\ b \end{bmatrix} = \frac{1}{\lambda_1 - \lambda_2} \begin{bmatrix} 1/\lambda_1^2 & -1/\lambda_2^2 \\ -\lambda_2/\lambda_1^2 & \lambda_1/\lambda_2^2 \end{bmatrix} \cdot \begin{bmatrix} g(\lambda_1) - g'(0)\lambda_1 - g(0) \\ g(\lambda_2) - g'(0)\lambda_2 - g(0) \end{bmatrix} \quad (9.7.13)$$

The minimum of the cubic (9.7.12) is at

$$\lambda = \frac{-b + \sqrt{b^2 - 3ag'(0)}}{3a} \quad (9.7.14)$$

We enforce that λ lie between $\lambda_{\max} = 0.5\lambda_1$ and $\lambda_{\min} = 0.1\lambda_1$.

The routine has two additional features, a minimum step length `alamin` and a maximum step length `stpmax`. `lnsrch` will also be used in the quasi-Newton minimization routine `dfpmin` in the next section.

```
template <class T>
void lnsrch(VecDoub_I &xold, const Doub fold, VecDoub_I &g, VecDoub_IO &p,
VecDoub_O &x, Doub &f, const Doub stpmax, Bool &check, T &func) {
    Given an n-dimensional point xold[0..n-1], the value of the function and gradient there, fold
    and g[0..n-1], and a direction p[0..n-1], finds a new point x[0..n-1] along the direction
    p from xold where the function or functor func has decreased "sufficiently." The new function
    value is returned in f. stpmax is an input quantity that limits the length of the steps so that you
    do not try to evaluate the function in regions where it is undefined or subject to overflow. p is
    usually the Newton direction. The output quantity check is false on a normal exit. It is true
    when x is too close to xold. In a minimization algorithm, this usually signals convergence and
    can be ignored. However, in a zero-finding algorithm the calling program should check whether
    the convergence is spurious.
    const Doub ALF=1.0e-4, TOLX=numeric_limits<Doub>::epsilon();
    ALF ensures sufficient decrease in function value; TOLX is the convergence criterion on Δx.

    Doub a,alam,alam2=0.0,alamin,b,disc,f2=0.0;
    Doub rhs1,rhs2,slope=0.0,sum=0.0,temp,test,tmplam;
    Int i,n=xold.size();
    check=false;
    for (i=0;i<n;i++) sum += p[i]*p[i];
    sum=sqrt(sum);
    if (sum > stpmax)
        for (i=0;i<n;i++)
```

roots_multidim.h

```

        p[i] *= stpmax/sum;
    for (i=0;i<n;i++)
        slope += g[i]*p[i];
    if (slope >= 0.0) throw("Roundoff problem in lnsrch.");
    test=0.0;
    for (i=0;i<n;i++) {
        temp=abs(p[i])/MAX(abs(xold[i]),1.0);
        if (temp > test) test=temp;
    }
    alamin=TOLX/test;
    alam=1.0;
    for (;;) {
        for (i=0;i<n;i++) x[i]=xold[i]+alam*p[i];
        f=func(x);
        if (alam < alamin) {
            for (i=0;i<n;i++) x[i]=xold[i];
            check=true;
            return;
        } else if (f <= fold+ALF*alam*slope) return;
        else {
            if (alam == 1.0)
                tmlam = -slope/(2.0*(f-fold-slope));
            else {
                rhs1=f-fold-alam*slope;
                rhs2=f2-fold-alam2*slope;
                a=(rhs1/(alam*alam)-rhs2/(alam2*alam2))/(alam-alam2);
                b=(-alam2*rhs1/(alam*alam)+alam*rhs2/(alam2*alam2))/(alam-alam2);
                if (a == 0.0) tmlam = -slope/(2.0*b);
                else {
                    disc=b*b-3.0*a*slope;
                    if (disc < 0.0) tmlam=0.5*alam;
                    else if (b <= 0.0) tmlam=(-b+sqrt(disc))/(3.0*a);
                    else tmlam=-slope/(b+sqrt(disc));
                }
                if (tmlam>0.5*alam)
                    tmlam=0.5*alam;
            }
            alam2=alam;
            f2 = f;
            alam=MAX(tmlam,0.1*alam);
        }
    }
}

```

Scale if attempted step is too big.

Compute $\lambda_{\min}$.

Always try full Newton step first.
Start of iteration loop.

Convergence on Δx . For zero finding, the calling program should verify the convergence.

Sufficient function decrease.
Backtrack.

First time.
Subsequent backtracks.

$\lambda \leq 0.5\lambda_1$.

$\lambda \geq 0.1\lambda_1$.
Try again.

9.7.2 Globally Convergent Newton Method

Using the above results on backtracking, here is the globally convergent Newton routine `newt` that uses `lnsrch`. A feature of `newt` is that you need not supply the Jacobian matrix analytically; the routine will attempt to compute the necessary partial derivatives of $\mathbf{F}$ by finite differences in the routine `NRfdjac`. This routine uses some of the techniques described in §5.7 for computing numerical derivatives. Of course, you can always replace `NRfdjac` with a routine that calculates the Jacobian analytically if this is easy for you to do.

The routine requires a user-supplied function or functor that computes the vector of functions to be zeroed. Its declaration as a function is

```
VecDoub vecfunc(VecDoub_I x);
```

(The name `vecfunc` is arbitrary.) The declaration as a functor is similar.

```
template <class T>
```

```
void newt(VecDoub_IO &x, Bool &check, T &vecfunc) {
```

Given an initial guess $x[0..n-1]$ for a root in n dimensions, find the root by a globally convergent Newton's method. The vector of functions to be zeroed, called $fvec[0..n-1]$ in the routine below, is returned by the user-supplied function or functor $vecfunc$ (see text). The output quantity $check$ is false on a normal return and true if the routine has converged to a local minimum of the function $fmin$ defined below. In this case try restarting from a different initial guess.

```
    const Int MAXITS=200;
```

```
    const Doub TOLF=1.0e-8,TOLMIN=1.0e-12,STPMX=100.0;
```

```
    const Doub TOLX=numeric_limits<Doub>::epsilon();
```

Here $MAXITS$ is the maximum number of iterations; $TOLF$ sets the convergence criterion on function values; $TOLMIN$ sets the criterion for deciding whether spurious convergence to a minimum of $fmin$ has occurred; $STPMX$ is the scaled maximum step length allowed in line searches; and $TOLX$ is the convergence criterion on $\delta \mathbf{x}$.

```
    Int i,j,its,n=x.size();
```

```
    Doub den,f,fold,stpmx,sum,temp,test;
```

```
    VecDoub g(n),p(n),xold(n);
```

```
    MatDoub fjac(n,n);
```

```
    NRfmin<T> fmin(vecfunc);
```

Set up NRfmin object.

```
    NRfdjac<T> fdjac(vecfunc);
```

Set up NRfdjac object.

```
    VecDoub &fvec=fmin.fvec;
```

Make an alias to simplify coding.

```
    f=fmin(x);
```

$fvec$ is also computed by this call.

```
    test=0.0;
```

Test for initial guess being a root. Use more stringent test than simply $TOLF$.

```
    for (i=0;i<n;i++)
        if (abs(fvec[i]) > test) test=abs(fvec[i]);
```

```
    if (test < 0.01*TOLF) {
```

```
        check=false;
```

```
        return;
```

```
    }
```

```
    sum=0.0;
```

```
    for (i=0;i<n;i++) sum += SQR(x[i]);
```

Calculate $stpmx$ for line searches.

```
    stpmx=STPMX*MAX(sqrt(sum),Doub(n));
```

```
    for (its=0;its<MAXITS;its++) {
```

Start of iteration loop.

```
        fjac=fdjac(x,fvec);
```

If analytic Jacobian is available, you can replace the struct `NRfdjac` below with your own struct.

```
        for (i=0;i<n;i++) {
```

Compute ∇f for the line search.

```
            sum=0.0;
```

```
            for (j=0;j<n;j++) sum += fjac[j][i]*fvec[j];
```

```
            g[i]=sum;
```

```
        }
```

```
        for (i=0;i<n;i++) xold[i]=x[i];
```

Store $\mathbf{x}$,

```
        fold=f;
```

and f .

```
        for (i=0;i<n;i++) p[i] = -fvec[i];
```

Right-hand side for linear equations.

```
        LUdcmp alu(fjac);
```

Solve linear equations by LU decomposition.

```
        alu.solve(p,p);
```

```
        lnsrch(xold,fold,g,p,x,f,stpmx,check,fmin);
```

`lnsrch` returns new $\mathbf{x}$ and f . It also calculates $fvec$ at the new $\mathbf{x}$ when it calls `fmin`.

```
        test=0.0;
```

Test for convergence on function values.

```
        for (i=0;i<n;i++)
```

```
            if (abs(fvec[i]) > test) test=abs(fvec[i]);
```

```
        if (test < TOLF) {
```

```
            check=false;
```

```
            return;
```

```
        }
```

```
        if (check) {
```

Check for gradient of f zero, i.e., spurious convergence.

```
            test=0.0;
```

```
            den=MAX(f,0.5*n);
```

```
            for (i=0;i<n;i++) {
```

```
                temp=abs(g[i])*MAX(abs(x[i]),1.0)/den;
```

```
                if (temp > test) test=temp;
```

```
            }
```

```
        }
```

```

        check=(test < TOLMIN);
        return;
    }
    test=0.0;
    for (i=0;i<n;i++) {
        temp=(abs(x[i]-xold[i]))/MAX(abs(x[i]),1.0);
        if (temp > test) test=temp;
    }
    if (test < TOLX)
        return;
}
throw("MAXITS exceeded in newt");
}

```

roots_multidim.h

```

template <class T>
struct NRfdjac {
    Computes forward-difference approximation to Jacobian.
    const Doub EPS;           Set to approximate square root of the machine pre-
    T &func;                   cision.
    NRfdjac(T &funcc) : EPS(1.0e-8),func(funcc) {}
    Initialize with user-supplied function or functor that returns the vector of functions to be
    zeroed.
    MatDoub operator() (VecDoub_I &x, VecDoub_I &fvec) {
        Returns the Jacobian array df[0..n-1][0..n-1]. On input, x[0..n-1] is the point at
        which the Jacobian is to be evaluated and fvec[0..n-1] is the vector of function values
        at the point.
        Int n=x.size();
        MatDoub df(n,n);
        VecDoub xh=x;
        for (Int j=0;j<n;j++) {
            Doub temp=xh[j];
            Doub h=EPS*abs(temp);
            if (h == 0.0) h=EPS;
            xh[j]=temp+h;           Trick to reduce finite-precision error.
            h=xh[j]-temp;
            VecDoub f=func(xh);
            xh[j]=temp;
            for (Int i=0;i<n;i++)    Forward difference formula.
                df[i][j]=(f[i]-fvec[i])/h;
        }
        return df;
    }
};

```

roots_multidim.h

```

template <class T>
struct NRfmin {
    Returns  $f = \frac{1}{2} \mathbf{F} \cdot \mathbf{F}$ . Also stores value of  $\mathbf{F}$  in fvec.
    VecDoub fvec;
    T &func;
    Int n;
    NRfmin(T &funcc) : func(funcc){}
    Initialize with user-supplied function or functor that returns the vector of functions to be
    zeroed.
    Doub operator() (VecDoub_I &x) {
        Returns  $f$  at  $x$ , and stores  $\mathbf{F}(x)$  in fvec.
        n=x.size();
        Doub sum=0;
        fvec=func(x);
        for (Int i=0;i<n;i++) sum += SQR(fvec[i]);
        return 0.5*sum;
    }
};

```

The routine `newt` assumes that the typical values of all components of $\mathbf{x}$ and of $\mathbf{F}$ are of order unity, and it can fail if this assumption is badly violated. You should rescale the variables by their typical values before invoking `newt` if this problem occurs.

9.7.3 Multidimensional Secant Methods: Broyden's Method

Newton's method as implemented above is quite powerful, but it still has several disadvantages. One drawback is that the Jacobian matrix is needed. In many problems analytic derivatives are unavailable. If function evaluation is expensive, then the cost of finite difference determination of the Jacobian can be prohibitive.

Just as the quasi-Newton methods to be discussed in §10.9 provide cheap approximations for the Hessian matrix in minimization algorithms, there are quasi-Newton methods that provide cheap approximations to the Jacobian for zero finding. These methods are often called *secant methods*, since they reduce to the secant method (§9.2) in one dimension (see, e.g., [2]). The best of these methods still seems to be the first one introduced, *Broyden's method* [3].

Let us denote the approximate Jacobian by $\mathbf{B}$. Then the i th quasi-Newton step $\delta\mathbf{x}_i$ is the solution of

$$\mathbf{B}_i \cdot \delta\mathbf{x}_i = -\mathbf{F}_i \quad (9.7.15)$$

where $\delta\mathbf{x}_i = \mathbf{x}_{i+1} - \mathbf{x}_i$ (cf. equation 9.7.3). The quasi-Newton or secant condition is that $\mathbf{B}_{i+1}$ satisfy

$$\mathbf{B}_{i+1} \cdot \delta\mathbf{x}_i = \delta\mathbf{F}_i \quad (9.7.16)$$

where $\delta\mathbf{F}_i = \mathbf{F}_{i+1} - \mathbf{F}_i$. This is the generalization of the one-dimensional secant approximation to the derivative, $\delta F / \delta x$. However, equation (9.7.16) does not determine $\mathbf{B}_{i+1}$ uniquely in more than one dimension.

Many different auxiliary conditions to pin down $\mathbf{B}_{i+1}$ have been explored, but the best-performing algorithm in practice results from Broyden's formula. This formula is based on the idea of getting $\mathbf{B}_{i+1}$ by making the least change to $\mathbf{B}_i$ consistent with the secant equation (9.7.16). Broyden showed that the resulting formula is

$$\mathbf{B}_{i+1} = \mathbf{B}_i + \frac{(\delta\mathbf{F}_i - \mathbf{B}_i \cdot \delta\mathbf{x}_i) \otimes \delta\mathbf{x}_i}{\delta\mathbf{x}_i \cdot \delta\mathbf{x}_i} \quad (9.7.17)$$

You can easily check that $\mathbf{B}_{i+1}$ satisfies (9.7.16).

Early implementations of Broyden's method used the Sherman-Morrison formula, equation (2.7.2), to invert equation (9.7.17) analytically,

$$\mathbf{B}_{i+1}^{-1} = \mathbf{B}_i^{-1} + \frac{(\delta\mathbf{x}_i - \mathbf{B}_i^{-1} \cdot \delta\mathbf{F}_i) \otimes \delta\mathbf{x}_i \cdot \mathbf{B}_i^{-1}}{\delta\mathbf{x}_i \cdot \mathbf{B}_i^{-1} \cdot \delta\mathbf{F}_i} \quad (9.7.18)$$

Then, instead of solving equation (9.7.3) by, e.g., *LU* decomposition, one determined

$$\delta\mathbf{x}_i = -\mathbf{B}_i^{-1} \cdot \mathbf{F}_i \quad (9.7.19)$$

by matrix multiplication in $O(N^2)$ operations. The disadvantage of this method is that it cannot easily be embedded in a globally convergent strategy, for which the gradient of equation (9.7.4) requires $\mathbf{B}$, not $\mathbf{B}^{-1}$,

$$\nabla(\frac{1}{2}\mathbf{F} \cdot \mathbf{F}) \simeq \mathbf{B}^T \cdot \mathbf{F} \quad (9.7.20)$$

Accordingly, we implement the update formula in the form (9.7.17).

However, we can still preserve the $O(N^2)$ solution of (9.7.3) by using *QR* decomposition (§2.10) instead of *LU* decomposition. The reason is that because of the special form of equation (9.7.17), the *QR* decomposition of $\mathbf{B}_i$ can be updated into the *QR* decomposition of $\mathbf{B}_{i+1}$ in $O(N^2)$ operations (§2.10). All we need is an initial approximation $\mathbf{B}_0$ to start the ball rolling. It is often acceptable to start simply with the identity matrix, and then allow $O(N)$

updates to produce a reasonable approximation to the Jacobian. We prefer to spend the first N function evaluations on a finite difference approximation to initialize $\mathbf{B}$ via a call to `NRfdjac`.

Since $\mathbf{B}$ is not the exact Jacobian, we are not guaranteed that $\delta\mathbf{x}$ is a descent direction for $f = \frac{1}{2}\mathbf{F} \cdot \mathbf{F}$ (cf. equation 9.7.5). Thus the line search algorithm can fail to return a suitable step if $\mathbf{B}$ wanders far from the true Jacobian. In this case, we reinitialize $\mathbf{B}$ by another call to `NRfdjac`.

Like the secant method in one dimension, Broyden's method converges superlinearly once you get close enough to the root. Embedded in a global strategy, it is almost as robust as Newton's method, and often needs far fewer function evaluations to determine a zero. Note that the final value of $\mathbf{B}$ is *not* always close to the true Jacobian at the root, even when the method converges.

The routine `broydn`, given below, is very similar to `newt` in organization. The principal differences are the use of *QR* decomposition instead of *LU*, and the updating formula instead of directly determining the Jacobian. The remarks at the end of `newt` about scaling the variables apply equally to `broydn`.

roots_multidim.h

```
template <class T>
void broydn(VecDoub_IO &x, Bool &check, T &vecfunc) {
    Given an initial guess x[0..n-1] for a root in n dimensions, find the root by Broyden's
    method embedded in a globally convergent strategy. The vector of functions to be zeroed,
    called fvec[0..n-1] in the routine below, is returned by the user-supplied function or functor
    vecfunc. The routines NRfdjac and NRfmin from newt are used. The output quantity check
    is false on a normal return and true if the routine has converged to a local minimum of the
    function fmin or if Broyden's method can make no further progress. In this case try restarting
    from a different initial guess.
    const Int MAXITS=200;
    const Doub EPS=numeric_limits<Doub>::epsilon();
    const Doub TOLF=1.0e-8, TOLX=EPS, STPMX=100.0, TOLMIN=1.0e-12;
    Here MAXITS is the maximum number of iterations; EPS is the machine precision; TOLF
    is the convergence criterion on function values; TOLX is the convergence criterion on  $\delta\mathbf{x}$ ;
    STPMX is the scaled maximum step length allowed in line searches; and TOLMIN is used to
    decide whether spurious convergence to a minimum of fmin has occurred.
    Bool restrt, skip;
    Int i, its, j, n=x.size();
    Doub den, f, fold, stpmax, sum, temp, test;
    VecDoub fvcold(n), g(n), p(n), s(n), t(n), w(n), xold(n);
    QRdcmp *qr;
    NRfmin<T> fmin(vecfunc);           Set up NRfmin object.
    NRfdjac<T> fdjac(vecfunc);        Set up NRfdjac object.
    VecDoub &fvec=fmin.fvec;          Make an alias to simplify coding.
    f=fmin(x);                        The vector fvec is also computed by this
    test=0.0;                          call.
    for (i=0; i<n; i++)                Test for initial guess being a root. Use more
        if (abs(fvec[i]) > test) test=abs(fvec[i]);    stringent test than simply
    if (test < 0.01*TOLF) {              TOLF.
        check=false;
        return;
    }
    for (sum=0.0, i=0; i<n; i++) sum += SQR(x[i]);    Calculate stpmax for line searches.
    stpmax=STPMX*MAX(sqrt(sum), Doub(n));
    restrt=true;
    for (its=1; its<=MAXITS; its++) {    Ensure initial Jacobian gets computed.
        Start of iteration loop.
        if (restrt) {                    Initialize or reinitialize Jacobian and QR de-
            qr=new QRdcmp(fdjac(x, fvec));    compose it.
            if (qr->sing) throw("singular Jacobian in broydn");
        } else {                        Carry out Broyden update.
            for (i=0; i<n; i++) s[i]=x[i]-xold[i];     $\mathbf{s} = \delta\mathbf{x}$ .
            for (i=0; i<n; i++) {                 $\mathbf{t} = \mathbf{R} \cdot \mathbf{s}$ .
                for (sum=0.0, j=i; j<n; j++) sum += qr->r[i][j]*s[j];
                t[i]=sum;
            }
            skip=true;
        }
    }
}
```



```

for (i=0;i<n;i++) {
    w =  $\delta \mathbf{F} - \mathbf{B} \cdot \mathbf{s}$ .
    for (sum=0.0,j=0;j<n;j++) sum += qr->qt[j][i]*t[j];
    w[i]=fvec[i]-fvcold[i]-sum;
    if (abs(w[i]) >= EPS*(abs(fvec[i])+abs(fvcold[i]))) skip=false;
    Don't update with noisy components of  $\mathbf{w}$ .
    else w[i]=0.0;
}
if (!skip) {
    qr->qtmult(w,t);
     $\mathbf{t} = \mathbf{Q}^T \cdot \mathbf{w}$ .
    for (den=0.0,i=0;i<n;i++) den += SQR(s[i]);
    for (i=0;i<n;i++) s[i] /= den;
    qr->update(t,s);
    Store  $\mathbf{s}/(\mathbf{s} \cdot \mathbf{s})$  in  $\mathbf{s}$ .
    Update  $\mathbf{R}$  and  $\mathbf{Q}^T$ .
    if (qr->sing) throw("singular update in broydn");
}
qr->qtmult(fvec,p);
for (i=0;i<n;i++)
    p[i] = -p[i];
for (i=n-1;i>=0;i--) {
    for (sum=0.0,j=0;j<=i;j++) sum -= qr->r[j][i]*p[j];
    g[i]=sum;
}
for (i=0;i<n;i++) {
    xold[i]=x[i];
    fvcold[i]=fvec[i];
}
fold=f;
qr->rsolve(p,p);
lnsrch(xold,fold,g,p,x,f,stpmax,check,fmin);
lnsrch returns new  $\mathbf{x}$  and  $f$ . It also calculates fvec at the new  $\mathbf{x}$  when it calls fmin.

test=0.0;
for (i=0;i<n;i++)
    if (abs(fvec[i]) > test) test=abs(fvec[i]);
if (test < TOLF) {
    check=false;
    delete qr;
    return;
}
if (check) {
    if (restrt) {
        delete qr;
        return;
    } else {
        test=0.0;
        den=MAX(f,0.5*n);
        for (i=0;i<n;i++) {
            temp=abs(g[i])*MAX(abs(x[i]),1.0)/den;
            if (temp > test) test=temp;
        }
        if (test < TOLMIN) {
            delete qr;
            return;
        }
        else restrt=true;
    }
} else {
    restrt=false;
    test=0.0;
    for (i=0;i<n;i++) {
        temp=(abs(x[i]-xold[i]))/MAX(abs(x[i]),1.0);
        if (temp > test) test=temp;
    }
}

```

Right-hand side for linear equations is $-\mathbf{Q}^T \cdot \mathbf{F}$.

Compute $\nabla f \approx (\mathbf{Q} \cdot \mathbf{R})^T \cdot \mathbf{F}$ for the line search.

Store $\mathbf{x}$ and $\mathbf{F}$.

Store f .

Solve linear equations.

lnsrch returns new $\mathbf{x}$ and f . It also calculates fvec at the new $\mathbf{x}$ when it calls fmin.

Test for convergence on function values.

True if line search failed to find a new $\mathbf{x}$.

Failure; already tried reinitializing the Jacobian.

Check for gradient of f zero, i.e., spurious convergence.

Try reinitializing the Jacobian.

Successful step; will use Broyden update for next step.

Test for convergence on $\delta \mathbf{x}$.

```

        if (test < TOLX) {
            delete qr;
            return;
        }
    }
    throw("MAXITS exceeded in broydn");
}

```

9.7.4 More Advanced Implementations

One of the principal ways that the methods described so far can fail is if $\mathbf{J}$ (in Newton's method) or $\mathbf{B}$ in (Broyden's method) becomes singular or nearly singular, so that $\delta\mathbf{x}$ cannot be determined. If you are lucky, this situation will not occur very often in practice. Methods developed so far to deal with this problem involve monitoring the condition number of $\mathbf{J}$ and perturbing $\mathbf{J}$ if singularity or near singularity is detected. This is most easily implemented if the QR decomposition is used instead of LU in Newton's method (see [2] for details). Our personal experience is that, while such an algorithm can solve problems where $\mathbf{J}$ is exactly singular and the standard Newton method fails, it is occasionally less robust on other problems where LU decomposition succeeds. Clearly implementation details involving roundoff, underflow, etc., are important here and the last word is yet to be written.

Our global strategies both for minimization and zero finding have been based on line searches. Other global algorithms, such as the *hook step* and *dogleg step* methods, are based instead on the *model-trust region* approach, which is related to the Levenberg-Marquardt algorithm for nonlinear least squares (§15.5). While somewhat more complicated than line searches, these methods have a reputation for robustness even when starting far from the desired zero or minimum [2].

CITED REFERENCES AND FURTHER READING:

- Deuffhard, P. 2004, *Newton Methods for Nonlinear Problems* (Berlin: Springer).[1]
 Dennis, J.E., and Schnabel, R.B. 1983, *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*; reprinted 1996 (Philadelphia: S.I.A.M.).[2]
 Broyden, C.G. 1965, "A Class of Methods for Solving Nonlinear Simultaneous Equations," *Mathematics of Computation*, vol. 19, pp. 577–593.[3]